

김석영_고객을 세그멘테이션하자 [프로젝트]

11-2. 데이터 불러오기

2.1 데이터 살펴보기

- 테이블에 있는 10개의 행만 출력하기

```
SELECT *  
FROM `project-3-mainquest-1.modulabs_project.data`  
LIMIT 10;
```

행	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
1	536363	81529A	WHITE HANGING HEART T-LAC...	6	2019-12-01 08:26:00 UTC	2.55	17850	United Kingdom
2	536363	71851	WHITE METAL LANTERN	6	2019-12-01 08:26:00 UTC	9.99	17850	United Kingdom
3	536365	844068	CREAM CUPID HEARTS COAT H...	8	2019-12-01 08:26:00 UTC	2.75	17850	United Kingdom
4	536365	842092	KNITTED UNION FLAG HOT MA...	6	2019-12-01 08:26:00 UTC	3.29	17850	United Kingdom
5	536364	842092	RED WOOLLY HOTTIE WHITE H...	6	2019-12-01 08:26:00 UTC	8.29	17850	United Kingdom
6	536365	21702	SET 7 BABUSHKA NESTING BO...	2	2019-12-01 08:26:00 UTC	7.65	17850	United Kingdom
7	536365	21709	GLASS STAR PROTECTED T-LIGHT...	6	2019-12-01 08:26:00 UTC	4.25	17850	United Kingdom
8	536366	22835	HARD WAREHAM UNION JACK	6	2019-12-01 08:26:00 UTC	1.85	17850	United Kingdom
9	536366	12832	HARD WAREHAM RED PLYAM DUT	6	2019-12-01 08:26:00 UTC	1.85	17850	United Kingdom
10	536367	84479	ADORNED COLOUR BIRD ORN...	12	2019-12-01 08:26:00 UTC	1.89	12047	United Kingdom

- 전체 데이터는 몇 행으로 구성되어 있는지 확인하기

```
SELECT COUNT(*) AS total_rows  
FROM `project-3-mainquest-1.modulabs_project.data`;
```

행	total_rows
1	541909

2.2 데이터 수 세기

- COUNT 함수를 사용해서, 각 컬럼별 데이터 포인트의 수를 세어 보기

```
SELECT  
COUNT(InvoiceNo) AS COUNT_InvoiceNo,  
COUNT(StockCode) AS COUNT_StockCode,  
COUNT(Description) AS COUNT_Description,  
COUNT(Quantity) AS COUNT_Quantity,  
COUNT(InvoiceDate) AS COUNT_InvoiceDate,  
COUNT(UnitPrice) AS COUNT_UnitPrice,  
COUNT(CustomerID) AS COUNT_CustomerID,  
COUNT(Country) AS COUNT_Country  
FROM `project-3-mainquest-1.modulabs_project.data`;
```

행	COUNT_InvoiceNo	COUNT_StockCode	COUNT_Description	COUNT_Quantity	COUNT_InvoiceDate	COUNT_UnitPrice	COUNT_CustomerID	COUNT_Country
1	541909	541909	540455	541909	541909	541909	406829	541909

11-4. 데이터 전처리(1): 결측치 제거

4.1 컬럼 별 누락된 값의 비율 계산

- 각 컬럼 별 누락된 값의 비율 계산

-- 컬럼의 결측치 비율을 계산하는 쿼리문 (예: InvoiceNo)

```
SELECT  
'InvoiceNo' AS column_name,
```

```
ROUND(
    SUM(CASE WHEN InvoiceNo IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100,
    2
) AS missing_percentage
FROM `project-3-mainquest-1.modulabs_project.data`;
```

행	column_name	missing_percenta...
1	InvoiceNo	0.0

- 계산된 각 컬럼의 누락 값을 UNION ALL을 통해 합치기

-- 방법 1

```
SELECT
    'InvoiceNo' AS column_name,
    ROUND(
        SUM(CASE WHEN InvoiceNo IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100,
        2
    ) AS missing_percentage
FROM `project-3-mainquest-1.modulabs_project.data`
```

UNION ALL

```
SELECT
    'StockCode' AS column_name,
    ROUND(
        SUM(CASE WHEN StockCode IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100,
        2
    ) AS missing_percentage
FROM `project-3-mainquest-1.modulabs_project.data`
```

UNION ALL

```
SELECT
    'Description' AS column_name,
    ROUND(
        SUM(CASE WHEN Description IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100,
        2
    ) AS missing_percentage
FROM `project-3-mainquest-1.modulabs_project.data`
```

UNION ALL

```
SELECT
    'Quantity' AS column_name,
    ROUND(
        SUM(CASE WHEN Quantity IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100,
        2
    ) AS missing_percentage
FROM `project-3-mainquest-1.modulabs_project.data`
```

UNION ALL

```
SELECT
    'InvoiceDate' AS column_name,
    ROUND(
        SUM(CASE WHEN InvoiceDate IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100,
        2
    ) AS missing_percentage
FROM `project-3-mainquest-1.modulabs_project.data`
```

UNION ALL

```

SELECT
  'UnitPrice' AS column_name,
  ROUND(
    SUM(CASE WHEN UnitPrice IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100,
    2
  ) AS missing_percentage
FROM `project-3-mainquest-1.modulabs_project.data`

UNION ALL

SELECT
  'CustomerID' AS column_name,
  ROUND(
    SUM(CASE WHEN CustomerID IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100,
    2
  ) AS missing_percentage
FROM `project-3-mainquest-1.modulabs_project.data`

UNION ALL

SELECT
  'Country' AS column_name,
  ROUND(
    SUM(CASE WHEN Country IS NULL THEN 1 ELSE 0 END) / COUNT(*) * 100,
    2
  ) AS missing_percentage
FROM `project-3-mainquest-1.modulabs_project.data`;

```

```

-- 방법 2

SELECT
  column_name,
  ROUND(
    (total - column_value) / total * 100,
    2
  )
FROM (
  SELECT
    'InvoiceNo' AS column_name,
    COUNT(InvoiceNo) AS column_value,
    COUNT(*) AS total
  FROM `project-3-mainquest-1.modulabs_project.data`

  UNION ALL

  SELECT
    'StockCode' AS column_name,
    COUNT(StockCode) AS column_value,
    COUNT(*) AS total
  FROM `project-3-mainquest-1.modulabs_project.data`

  UNION ALL

  SELECT
    'Description' AS column_name,
    COUNT(Description) AS column_value,
    COUNT(*) AS total
  FROM `project-3-mainquest-1.modulabs_project.data`

  UNION ALL

  SELECT
    'Quantity' AS column_name,

```

```

COUNT(Quantity) AS column_value,
COUNT(*) AS total
FROM `project-3-mainquest-1.modulabs_project.data`

UNION ALL

SELECT
'InvoiceDate' AS column_name,
COUNT(InvoiceDate) AS column_value,
COUNT(*) AS total
FROM `project-3-mainquest-1.modulabs_project.data`

UNION ALL

SELECT
'UnitPrice' AS column_name,
COUNT(UnitPrice) AS column_value,
COUNT(*) AS total
FROM `project-3-mainquest-1.modulabs_project.data`

UNION ALL

SELECT
'CustomerID' AS column_name,
COUNT(CustomerID) AS column_value,
COUNT(*) AS total
FROM `project-3-mainquest-1.modulabs_project.data`

UNION ALL

SELECT
'Country' AS column_name,
COUNT(Country) AS column_value,
COUNT(*) AS total
FROM `project-3-mainquest-1.modulabs_project.data`
) AS column_data;

```

행	column_name	missing_percenta...
1	InvoiceNo	0.0
2	StockCode	0.0
3	Description	0.27
4	Quantity	0.0
5	InvoiceDate	0.0
6	UnitPrice	0.0
7	CustomerID	24.93
8	Country	0.0

4.2 결측치 처리 전략

- `StockCode = '85123A'` 의 `Description` 을 추출하는 쿼리문을 작성하기

```

SELECT DISTINCT
Description
FROM `project-3-mainquest-1.modulabs_project.data`
WHERE StockCode = '85123A';

```

행	Description
1	WHITE HANGING HEART T-LIGHT HOLDER
2	?
3	wrongly marked carton 22804
4	CREAM HANGING HEART T-LIGHT HOLDER

4.3 결측치 처리

- DELETE 구문을 사용하며, WHERE 절을 통해 데이터를 제거할 조건을 제시

-- 결측치 확인

```
SELECT
  COUNT(*) AS missing_rows
FROM `project-3-mainquest-1.modulabs_project.data`
WHERE CustomerID IS NULL
   OR Description IS NULL;
```

행	missing_rows
1	135080

-- 결측치 제거

```
DELETE
FROM `project-3-mainquest-1.modulabs_project.data`
WHERE CustomerID IS NULL
   OR Description IS NULL;
```

쿼리 결과

결과 저장 다음에서 열기

작업 정보 결과 실행 세부정보 실행 그래프

이 문으로 data의 행 135,080개가 삭제되었습니다.

테이블로 이동

-- 결측치 남아 있는지 확인

```
SELECT
  COUNT(*) AS remaining_invalid_rows
FROM `project-3-mainquest-1.modulabs_project.data`
WHERE CustomerID IS NULL
   OR Description IS NULL;
```

행	remaining_invalid...
1	0

11-5. 데이터 전처리(2): 중복값 처리

5.1 중복값 확인

- 중복된 행의 수 세어보기

-- 8개의 컬럼에 그룹 함수를 적용한 후,
-- COUNT가 1보다 큰 데이터를 세어보기

```

SELECT
  COUNT(*) AS duplicated_rows
FROM (
  SELECT
    InvoiceNo,
    StockCode,
    Description,
    Quantity,
    InvoiceDate,
    UnitPrice,
    CustomerID,
    Country,
    COUNT(*) AS cnt
  FROM `project-3-mainquest-1.modulabs_project.data`
  GROUP BY
    InvoiceNo,
    StockCode,
    Description,
    Quantity,
    InvoiceDate,
    UnitPrice,
    CustomerID,
    Country
  HAVING COUNT(*) > 1
);

```

행	duplicated_rows
1	4837

5.2 중복값 처리

- 중복값을 제거하는 쿼리문 작성하기

```

-- CREATE OR REPLACE TABLE 구문을 활용하여
-- 모든 컬럼(*)을 DISTINCT 한 데이터로 업데이트

CREATE OR REPLACE TABLE `project-3-mainquest-1.modulabs_project.data` AS
SELECT DISTINCT
  *
FROM `project-3-mainquest-1.modulabs_project.data`;

```

-- 전체 행의 수 확인

```

SELECT
  COUNT(*) AS total_rows_after_dedup
FROM `project-3-mainquest-1.modulabs_project.data`;

```

행	total_rows_after_dedup
1	401604

-- 중복 재확인 (0이 나오면 정상)

```

SELECT
  COUNT(*) AS duplicated_rows
FROM (
  SELECT
    InvoiceNo,
    StockCode,

```

```

Description,
Quantity,
InvoiceDate,
UnitPrice,
CustomerID,
Country,
COUNT(*) AS cnt
FROM `project-3-mainquest-1.modulabs_project.data`
GROUP BY
InvoiceNo,
StockCode,
Description,
Quantity,
InvoiceDate,
UnitPrice,
CustomerID,
Country
HAVING COUNT(*) > 1
);

```

행	duplicated_rows
1	0

11-6. 데이터 전처리(3): 오류값 처리

6.1 InvoiceNo 살펴보기

- 고유(unique)한 InvoiceNo 의 개수를 출력하기

```

SELECT
COUNT(DISTINCT InvoiceNo) AS unique_invoice_count
FROM `project-3-mainquest-1.modulabs_project.data`;

```

행	unique_invoice_count
1	22190

- 고유한 InvoiceNo 를 앞에서부터 100개를 출력하기

```

SELECT DISTINCT
InvoiceNo
FROM `project-3-mainquest-1.modulabs_project.data`
LIMIT 100;

```

행	InvoiceNo
1	541431
2	C541433
3	537626
4	542237
5	549222
6	556201
7	562032
8	573511
9	581180

- InvoiceNo 가 'C'로 시작하는 행을 필터링 할 수 있는 쿼리문을 작성하기 (100행까지만 출력)

```
SELECT
*
FROM `project-3-mainquest-1.modulabs_project.data`
WHERE InvoiceNo LIKE 'C%'
LIMIT 100;
```

#	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
1	CS41433	22166	MEDIUM CERAMIC TOP STORA...	-74016	2011-01-18 16:17:00 UTC	1.04	12386	United Kingdom
2	CS43205	84	Manual	-1	2011-03-01 16:47:00 UTC	182.75	12392	Norway
3	CS43205	84	Manual	-1	2011-03-01 16:47:00 UTC	280.00	12392	Norway
4	CS43300	84	Manual	-1	2011-03-01 16:49:00 UTC	376.5	12392	Norway
5	CS47380	44032	PINK HEART SHAPED EGG PRIN...	-12	2011-03-22 16:07:00 UTC	1.65	12392	Norway
6	CS47380	21914	BULL HAMBURGERS IN BAGS	-12	2011-03-22 16:07:00 UTC	1.28	12392	Norway
7	CS47380	21948	CERAMIC CAGE DESIGN SPOT...	-12	2011-03-22 16:07:00 UTC	1.49	12392	Norway
8	CS47380	22045	CERAMIC HEART PARRY CAGE...	-12	2011-03-22 16:07:00 UTC	1.45	12392	Norway

- 구매 건 상태가 Canceled 인 데이터의 비율(%) - 소수점 첫번째 자리까지

```
SELECT
ROUND(
SUM(CASE WHEN InvoiceNo LIKE 'C%' THEN 1 ELSE 0 END) / COUNT(*) * 100,
1
) AS canceled_percentage
FROM `project-3-mainquest-1.modulabs_project.data`;
```

행	canceled_percentage
1	2.2

6.2 StockCode 살펴보기

- 고유한 StockCode 의 개수를 출력하기

```
SELECT
COUNT(DISTINCT StockCode) AS unique_stockcode_count
FROM `project-3-mainquest-1.modulabs_project.data`;
```

행	unique_stockcode_count
1	3684

- 어떤 제품이 가장 많이 판매되었는지 보기 위하여 StockCode 별 등장 빈도를 출력하기

-- 상위 10개의 제품들 출력

```
SELECT
StockCode,
COUNT(*) AS sell_cnt
FROM `project-3-mainquest-1.modulabs_project.data`
GROUP BY StockCode
ORDER BY sell_cnt DESC
LIMIT 10;
```


행	StockCode	sell_cnt
1	85123A	2065
2	22423	1894
3	85099B	1659
4	47566	1409
5	84879	1405
6	20725	1346
7	22720	1224
8	POST	1196
9	22197	1110
10	23203	1108

- **StockCode** 컬럼의 값 중 비정상 코드 식별 및 제거

-- StockCode의 문자열 내 숫자의 길이를 구하기

```
WITH UniqueStockCodes AS (
  SELECT DISTINCT
    StockCode
  FROM `project-3-mainquest-1.modulabs_project.data`
)
SELECT
  LENGTH(StockCode)
  - LENGTH(REGEXP_REPLACE(StockCode, r'[0-9]', '')) AS number_count,
  COUNT(*) AS stock_cnt
FROM UniqueStockCodes
GROUP BY number_count
ORDER BY stock_cnt DESC;
```

행	number_count	stock_cnt
1	5	3676
2	0	7
3	1	1

-- 숫자가 0~1개인 값들에는 어떤 코드들이 들어가 있는지를 확인

```
SELECT DISTINCT
  StockCode,
  number_count
FROM (
  SELECT
    StockCode,
    LENGTH(StockCode)
    - LENGTH(REGEXP_REPLACE(StockCode, r'[0-9]', '')) AS number_count
  FROM `project-3-mainquest-1.modulabs_project.data`
)
WHERE number_count IN (0, 1);
```

행	StockCode	number_count
1	POST	0
2	M	0
3	C2	1
4	D	0
5	BANK CHARGES	0
6	PADS	0
7	DOT	0
8	CRUK	0

-- 해당 코드 값들을 가지고 있는 데이터 수의 전체 데이터에서 차지하는 비중 구하기 (소수점 두번째 자리까지)

```
SELECT
  ROUND(
    SUM(
      CASE
        WHEN LENGTH(StockCode)
          - LENGTH(REGEXP_REPLACE(StockCode, r'[0-9]', '')) IN (0, 1)
        THEN 1
        ELSE 0
      END
    ) / COUNT(*) * 100,
    2
  ) AS special_stockcode_percentage
FROM `project-3-mainquest-1.modulabs_project.data`;
```

행	special_stockcode_percentage
1	0.48

- 제품과 관련되지 않은 거래 기록 제거하기

-- 삭제 전 확인

```
SELECT
  COUNT(*) AS rows_to_delete
FROM `project-3-mainquest-1.modulabs_project.data`
WHERE StockCode IN (
  SELECT DISTINCT
    StockCode
  FROM (
    SELECT
      StockCode,
      LENGTH(StockCode)
        - LENGTH(REGEXP_REPLACE(StockCode, r'[0-9]', '')) AS number_count
    FROM `project-3-mainquest-1.modulabs_project.data`
  )
  WHERE number_count IN (0, 1)
);
```

행	rows_to_delete
1	1915

-- 삭제

```
DELETE
FROM `project-3-mainquest-1.modulabs_project.data`
WHERE StockCode IN (
  SELECT DISTINCT
    StockCode
  FROM (
    SELECT
      StockCode,
      LENGTH(StockCode)
        - LENGTH(REGEXP_REPLACE(StockCode, r'[0-9]', '')) AS number_count
    FROM `project-3-mainquest-1.modulabs_project.data`
  )
  WHERE number_count IN (0, 1)
);
```

쿼리 결과	결과 저장	다음에서 열기
작업 정보	결과	실행 세부정보 실행 그래프
이 문으로 data의 행 1,915개가 삭제되었습니다.		
테이블로 이동		

-- 삭제 후 검증

```
SELECT
  COUNT(*) AS remaining_non_product_rows
FROM `project-3-mainquest-1.modulabs_project.data`
WHERE LENGTH(StockCode)
  - LENGTH(REGEXP_REPLACE(StockCode, r'[0-9]', '')) IN (0, 1);
```

행	remaining_non_product_rows
1	0

6.3 Description 살펴보기

- 고유한 Description 별 출현 빈도를 계산하고 상위 30개를 출력하기

```
SELECT
  Description,
  COUNT(*) AS description_cnt
FROM `project-3-mainquest-1.modulabs_project.data`
GROUP BY Description
ORDER BY description_cnt DESC
LIMIT 30;
```

행	Description	description_cnt
1	WHITE HANGING HEART T-LIGHT HOLDER	2058
2	REGENCY CAKESTAND 3 TIER	1894
3	JUMBO BAG RED RETROSPOT	1659
4	PARTY BUNTING	1409
5	ASSORTED COLOUR BIRD ORNAMENT	1405
6	LUNCH BAG RED RETROSPOT	1345
7	SET OF 3 CAKE TINS PANTRY DESIGN	1224
8	LUNCH BAG BLACK SKULL	1099
9	PACK OF 72 RETROSPOT CAKE CASES	1062

- 대소문자가 혼합된 Description 이 있는지 확인

```
SELECT DISTINCT
  Description
FROM `project-3-mainquest-1.modulabs_project.data`
WHERE REGEXP_CONTAINS(Description, r'[a-z]');
```

행	Description
1	BAG 250g SWIRLY MARBLES
2	3 TRADITIONAL BISCUIT CUTTERS SET
3	BAG 125g SWIRLY MARBLES
4	POLYESTER FILLER PAD 30CMx30CM
5	BAG 500g SWIRLY MARBLES
6	POLYESTER FILLER PAD 45x45cm
7	POLYESTER FILLER PAD 40x40cm
8	ESSENTIAL BALM 3.5g TIN IN ENVELOPE
9	High Resolution Image
10	FRENCH BLUE METAL DOOR SIGN No
11	POLYESTER FILLER PAD 45x30cm
12	Next Day Carriage
13	NUMBER TILE VINTAGE FONT No
14	NUMBER TILE COTTAGE GARDEN No
15	POLYESTER FILLER PAD 65CMx65CM
16	FLOWERS HANDBAG blue and orange
17	FOLK ART GREETING CARD,pack/12
18	THE KING GIFT BAG 25x24x12cm
19	POLYESTER FILLER PAD 60x40cm

- 서비스 관련 정보를 포함하는 행들 제거하기

-- 삭제 전 확인

```
SELECT
  COUNT(*) AS service_rows
FROM `project-3-mainquest-1.modulabs_project.data`
WHERE Description LIKE '%Next Day Carriage%'
  OR Description LIKE '%High Resolution Image%';
```

행	service_rows
1	83

-- 삭제

```
DELETE
FROM `project-3-mainquest-1.modulabs_project.data`
WHERE Description LIKE '%Next Day Carriage%'
  OR Description LIKE '%High Resolution Image%';
```

-- 삭제 후 검증

```
SELECT
  COUNT(*) AS service_rows
FROM `project-3-mainquest-1.modulabs_project.data`
WHERE Description LIKE '%Next Day Carriage%'
  OR Description LIKE '%High Resolution Image%';
```

행	service_rows
1	0

- 대소문자를 혼합하고 있는 데이터를 대문자로 표준화하기

```
CREATE OR REPLACE TABLE `project-3-mainquest-1.modulabs_project.data` AS
SELECT
  * EXCEPT (Description),
  UPPER(Description) AS Description
FROM `project-3-mainquest-1.modulabs_project.data`;
```

-- 소문자 포함 여부 재확인

```
SELECT
  COUNT(*) AS lowercase_rows
FROM `project-3-mainquest-1.modulabs_project.data`
WHERE REGEXP_CONTAINS(Description, r'[a-z]');
```

행	lowercase_rows
1	0

6.4 UnitPrice 살펴보기

- UnitPrice 의 최소값, 최대값, 평균을 구하기

```
SELECT
  MIN(UnitPrice) AS min_price,
  MAX(UnitPrice) AS max_price,
  AVG(UnitPrice) AS avg_price
FROM `project-3-mainquest-1.modulabs_project.data`;
```

행	min_price	max_price	avg_price
1	0.0	649.5	2.9049567574059876

- 단가가 0원인 거래의 개수, 구매 수량(Quantity)의 최소값, 최대값, 평균 구하기

```
SELECT
  COUNT(*) AS cnt_quantity,
  MIN(Quantity) AS min_quantity,
  MAX(Quantity) AS max_quantity,
  AVG(Quantity) AS avg_quantity
FROM `project-3-mainquest-1.modulabs_project.data`
WHERE UnitPrice = 0;
```

행	cnt_quantity	min_quantity	max_quantity	avg_quantity
1	33	1	12540	420.51515151515139

- UnitPrice = 0 를 제거하고 일관된 데이터셋을 유지하기

```
CREATE OR REPLACE TABLE `project-3-mainquest-1.modulabs_project.data` AS
SELECT
  *
FROM `project-3-mainquest-1.modulabs_project.data`
WHERE UnitPrice <> 0;
```

-- 확인용 쿼리

```
SELECT COUNT(*) AS zero_price_rows
```

```
FROM `project-3-mainquest-1.modulabs_project.data`
WHERE UnitPrice = 0;
```

행	zero_price_rows
1	0

11-7. RFM 스코어

7.1 Recency

- **InvoiceDate** 컬럼을 연월일 자료형으로 변경하기

```
SELECT
  DATE(InvoiceDate) AS InvoiceDay,
  *
FROM `project-3-mainquest-1.modulabs_project.data`;
```

#	InvoiceDay	InvoiceNo	StockCode	Quantity	InvoiceDate	UnitPrice	CustomerID	Country	Description
1	2011-01-18	541481	22166	74215	2011-01-18 10:01:00 UTC	1.04	13346	United Kingdom	MEDIUM CERAMIC TOP STORAGE JAR
2	2011-01-18	541482	22166	74215	2011-01-18 10:17:00 UTC	1.04	13346	United Kingdom	MEDIUM CERAMIC TOP STORAGE JAR
3	2010-12-07	557626	20760	12	2010-12-07 14:57:00 UTC	4.45	13347	Ireland	BLACK EAR MUFF HEADPHONES
4	2010-12-07	557626	22726	4	2010-12-07 14:57:00 UTC	3.75	13347	Ireland	ALARM CLOCK BAKELITE GREEN
5	2010-12-07	557626	84658A	24	2010-12-07 14:57:00 UTC	2.95	13347	Ireland	80-000 PICTURE PLAYING CARDS
6	2010-12-07	557626	22771	12	2010-12-07 14:57:00 UTC	1.25	13347	Ireland	CLEAR DRAWER KNICK ACRYLIC EDWARDIAN
7	2010-12-07	557626	22467	4	2010-12-07 14:57:00 UTC	4.25	13347	Ireland	SET OF 2 TWO HANDLE BATHROOM
8	2010-12-07	557626	22725	4	2010-12-07 14:57:00 UTC	3.75	13347	Ireland	ALARM CLOCK BAKELITE CHOCOLATE
9	2010-12-07	557626	22725	4	2010-12-07 14:57:00 UTC	3.75	13347	Ireland	ALARM CLOCK BAKELITE CHOCOLATE
9	2010-12-07	557626	85116	12	2010-12-07 14:57:00 UTC	2.1	13347	Ireland	BLACK CANDLEABRA TIGHT-HOLDER

- 가장 최근 구매 일자를 MAX() 함수로 찾아보기

```
SELECT
  MAX(DATE(InvoiceDate)) OVER () AS most_recent_date,
  DATE(InvoiceDate) AS InvoiceDay,
  *
FROM `project-3-mainquest-1.modulabs_project.data`;
```

#	most_recent_date	InvoiceDay	InvoiceNo	StockCode	Quantity	InvoiceDate	UnitPrice	CustomerID	Country	Description
1	2011-12-09	2011-01-18	541481	22166	74215	2011-01-18 10:01:00 UTC	1.04	13346	United Kingdom	MEDIUM CERAMIC TOP STORAGE JAR
2	2011-12-09	2011-01-18	541482	22166	74215	2011-01-18 10:17:00 UTC	1.04	13346	United Kingdom	MEDIUM CERAMIC TOP STORAGE JAR
3	2011-12-09	2010-12-07	557626	20760	12	2010-12-07 14:57:00 UTC	4.45	13347	Ireland	BLACK EAR MUFF HEADPHONES
4	2011-12-09	2010-12-07	557626	22726	4	2010-12-07 14:57:00 UTC	3.75	13347	Ireland	ALARM CLOCK BAKELITE GREEN
5	2011-12-09	2010-12-07	557626	84658A	24	2010-12-07 14:57:00 UTC	2.95	13347	Ireland	80-000 PICTURE PLAYING CARDS
6	2011-12-09	2010-12-07	557626	22771	12	2010-12-07 14:57:00 UTC	1.25	13347	Ireland	CLEAR DRAWER KNICK ACRYLIC EDWARDIAN
7	2011-12-09	2010-12-07	557626	22467	4	2010-12-07 14:57:00 UTC	4.25	13347	Ireland	SET OF 2 TWO HANDLE BATHROOM
8	2011-12-09	2010-12-07	557626	22725	4	2010-12-07 14:57:00 UTC	3.75	13347	Ireland	ALARM CLOCK BAKELITE CHOCOLATE
9	2011-12-09	2010-12-07	557626	85116	12	2010-12-07 14:57:00 UTC	2.1	13347	Ireland	BLACK CANDLEABRA TIGHT-HOLDER
10	2011-12-09	2010-12-07	557626	20760	6	2010-12-07 14:57:00 UTC	5.49	13347	Ireland	CANDYPLATE EAR MUFF HEADPHONES

- 유저 별로 가장 큰 InvoiceDay를 찾아서 가장 최근 구매일로 저장하기

```
SELECT
  CustomerID,
  MAX(DATE(InvoiceDate)) AS InvoiceDay
FROM `project-3-mainquest-1.modulabs_project.data`
GROUP BY CustomerID;
```

행	CustomerID	InvoiceDay
1	12346	2011-01-18
2	12347	2011-12-07
3	12348	2011-09-25
4	12349	2011-11-21
5	12350	2011-02-02
6	12352	2011-11-03
7	12353	2011-05-19
8	12354	2011-04-21
9	12355	2011-05-09
10	12356	2011-11-17

- 가장 최근 일자(`most_recent_date`)와 유저별 마지막 구매일(`InvoiceDay`)간의 차이를 계산하기

-- 방법 1

```
SELECT
  CustomerID,
  EXTRACT(DAY FROM MAX(InvoiceDay) OVER () - InvoiceDay) AS recency
FROM (
  SELECT
    CustomerID,
    MAX(DATE(InvoiceDate)) AS InvoiceDay
  FROM `project-3-mainquest-1.modulabs_project.data`
  GROUP BY CustomerID
);
```

-- 방법 2

```
SELECT
  CustomerID,
  DATE_DIFF(MAX(InvoiceDay) OVER (), InvoiceDay, DAY) AS recency
FROM (
  SELECT
    CustomerID,
    MAX(DATE(InvoiceDate)) AS InvoiceDay
  FROM `project-3-mainquest-1.modulabs_project.data`
  GROUP BY CustomerID
);
```

행	CustomerID	recency
1	12346	325
2	12347	2
3	12348	75
4	12349	18
5	12350	310
6	12352	36
7	12353	204
8	12354	232
9	12355	214
10	12356	22

- 최종 데이터 셋에 필요한 데이터들을 각각 정제해서 이어붙이고 지금까지의 결과를 `user_r` 이라는 이름의 테이블로 저장하기

```
CREATE OR REPLACE TABLE `project-3-mainquest-1.modulabs_project.user_r` AS
SELECT
```

```

CustomerID,
DATE_DIFF(MAX(InvoiceDay) OVER (), InvoiceDay, DAY) AS recency
FROM (
SELECT
CustomerID,
MAX(InvoiceDate) AS InvoiceDay
FROM `project-3-mainquest-1.modulabs_project.data`
GROUP BY CustomerID
);

```

-- 테이블 생성 확인

```

SELECT
*
FROM `project-3-mainquest-1.modulabs_project.user_r`;

```

행	CustomerID	recency
1	13069	0
2	17490	0
3	17001	0
4	12662	0
5	15694	0
6	13777	0
7	15311	0
8	16626	0
9	17364	0
10	18102	0

7.2 Frequency

- 고객마다 고유한 InvoiceNo의 수를 세어보기

```

SELECT
CustomerID,
COUNT(DISTINCT InvoiceNo) AS purchase_cnt
FROM `project-3-mainquest-1.modulabs_project.data`
GROUP BY CustomerID;

```

행	CustomerID	purchase_cnt
1	12346	2
2	12347	7
3	12348	4
4	12349	1
5	12350	1
6	12352	8
7	12353	1
8	12354	1
9	12355	1
10	12356	3

- 각 고객 별로 구매한 아이템의 총 수량 더하기

```

SELECT
CustomerID,
SUM(Quantity) AS item_cnt

```



```
FROM `project-3-mainquest-1.modulabs_project.data`
GROUP BY CustomerID;
```

행	CustomerID	item_cnt
1	12346	0
2	12347	2458
3	12348	2332
4	12349	630
5	12350	196
6	12352	463
7	12353	20
8	12354	530
9	12355	240
10	12356	1573

- 전체 거래 건수 계산과 구매한 아이템의 총 수량 계산의 결과를 합쳐서 `user_rf` 라는 이름의 테이블에 저장하기

```
CREATE OR REPLACE TABLE `project-3-mainquest-1.modulabs_project.user_rf` AS
```

```
-- (1) 전체 거래 건수 계산
```

```
WITH purchase_cnt AS (
  SELECT
    CustomerID,
    COUNT(DISTINCT InvoiceNo) AS purchase_cnt
  FROM `project-3-mainquest-1.modulabs_project.data`
  GROUP BY CustomerID
),
```

```
-- (2) 구매한 아이템 총 수량 계산
```

```
item_cnt AS (
  SELECT
    CustomerID,
    SUM(Quantity) AS item_cnt
  FROM `project-3-mainquest-1.modulabs_project.data`
  GROUP BY CustomerID
)
```

```
-- 기존의 user_r에 (1)과 (2)를 통합
```

```
SELECT
  pc.CustomerID,
  pc.purchase_cnt,
  ic.item_cnt,
  ur.recentry
FROM purchase_cnt AS pc
JOIN item_cnt AS ic
  ON pc.CustomerID = ic.CustomerID
JOIN `project-3-mainquest-1.modulabs_project.user_r` AS ur
  ON pc.CustomerID = ur.CustomerID;
```

```
-- 테이블 생성 확인
```

```
SELECT
  *
FROM `project-3-mainquest-1.modulabs_project.user_rf`;
```

행	CustomerID	purchase_cnt	item_cnt	recency
1	12713	1	505	0
2	13436	1	76	1
3	13298	1	96	1
4	14569	1	79	1
5	15520	1	314	1
6	14204	1	72	2
7	15471	1	256	2
8	15195	1	1404	2
9	14578	1	240	3
10	17914	1	457	3

7.3 Monetary

- 고객별 총 지출액 계산 (소수점 첫째 자리에서 반올림)

```
SELECT
  CustomerID,
  ROUND(SUM(Quantity * UnitPrice), 1) AS user_total
FROM `project-3-mainquest-1.modulabs_project.data`
GROUP BY CustomerID;
```

행	CustomerID	user_total
1	12346	0.0
2	12347	4310.0
3	12348	1437.2
4	12349	1457.5
5	12350	294.4
6	12352	1265.4
7	12353	89.0
8	12354	1079.4
9	12355	459.4
10	12356	2487.4

- 고객별 평균 거래 금액 계산

```
-- 1) data 테이블을 user_rf 테이블과 조인(LEFT JOIN)한 후,
-- 2) purchase_cnt로 나누어서
-- 3) user_rfm 테이블로 저장하기
```

```
CREATE OR REPLACE TABLE `project-3-mainquest-1.modulabs_project.user_rfm` AS
SELECT
  rf.CustomerID AS CustomerID,
  rf.purchase_cnt,
  rf.item_cnt,
  rf.recency,
  ut.user_total,
  ROUND(ut.user_total / rf.purchase_cnt, 1) AS user_average
FROM `project-3-mainquest-1.modulabs_project.user_rf` rf
LEFT JOIN (
  -- 고객 별 총 지출액
  SELECT
    CustomerID,
    ROUND(SUM(Quantity * UnitPrice), 1) AS user_total
  FROM `project-3-mainquest-1.modulabs_project.data`
  GROUP BY CustomerID
```

```
) ut
ON rf.CustomerID = ut.CustomerID;
```

-- 테이블 생성 확인

```
SELECT
*
FROM `project-3-mainquest-1.modulabs_project.user_rfm`;
```

행	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average
1	12713	1	505	0	794.5	794.5
2	14569	1	79	1	227.4	227.4
3	13436	1	76	1	196.9	196.9
4	15520	1	314	1	343.5	343.5
5	13298	1	96	1	360.0	360.0
6	14204	1	72	2	150.6	150.6
7	15195	1	1404	2	3861.0	3861.0
8	15471	1	256	2	454.5	454.5
9	12442	1	181	3	144.1	144.1
10	15992	1	17	3	42.0	42.0

7.4 RFM 통합 테이블 출력하기

- 최종 user_rfm 테이블을 출력하기

-- 테이블 생성 확인

```
SELECT
*
FROM `project-3-mainquest-1.modulabs_project.user_rfm`;
```

행	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average
1	12713	1	505	0	794.5	794.5
2	14569	1	79	1	227.4	227.4
3	13436	1	76	1	196.9	196.9
4	15520	1	314	1	343.5	343.5
5	13298	1	96	1	360.0	360.0
6	14204	1	72	2	150.6	150.6
7	15195	1	1404	2	3861.0	3861.0
8	15471	1	256	2	454.5	454.5
9	12442	1	181	3	144.1	144.1
10	15992	1	17	3	42.0	42.0

11-8. 추가 Feature 추출

8.1 구매하는 제품의 다양성

- 고객들의 제품 구매 행동 속 구매 제품의 다양성 살펴보기

- 1) 고객 별로 구매한 상품들의 고유한 수를 계산하기
- 2) user_rfm 테이블과 결과를 합치기
- 3) user_data라는 이름의 테이블에 저장하기

```
CREATE OR REPLACE TABLE `project-3-mainquest-1.modulabs_project.user_data` AS
WITH unique_products AS (
SELECT
CustomerID,
COUNT(DISTINCT StockCode) AS unique_products
FROM `project-3-mainquest-1.modulabs_project.data`
GROUP BY CustomerID
)
SELECT
```

```

ur.*
up.* EXCEPT (CustomerID)
FROM `project-3-mainquest-1.modulabs_project.user_rfm` AS ur
JOIN unique_products AS up
ON ur.CustomerID = up.CustomerID;

```

-- 테이블 생성 확인

```

SELECT
*
FROM `project-3-mainquest-1.modulabs_project.user_data`;

```

명	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average	unique_products
1	13099	1	288	99	207.4	207.4	1
2	12943	1	-1	301	-3.8	-3.8	1
3	15316	1	100	326	165.0	165.0	1
4	13991	1	4	203	59.8	59.8	1
5	13829	1	-12	359	-102.0	-102.0	1
6	16454	1	2	64	5.9	5.9	1
7	16990	1	100	218	179.0	179.0	1
8	13366	1	144	50	56.2	56.2	1
9	16323	1	50	196	207.5	207.5	1
10	18133	1	1350	212	931.5	931.5	1

8.2 평균 구매 주기

- 고객들의 쇼핑 패턴을 이해하는 것을 목표로 고객 별 재방문 주기 살펴보기

-- 평균 구매 소요 일수를 계산하고, 그 결과를 user_data에 통합

```

CREATE OR REPLACE TABLE `project-3-mainquest-1.modulabs_project.user_data` AS
WITH purchase_intervals AS (
  -- (2) 고객별 구매와 구매 사이의 평균 소요 일수
  SELECT
    CustomerID,
    IFNULL(ROUND(AVG(interval_), 2), 0) AS average_interval
  FROM (
    -- (1) 구매와 구매 사이에 소요된 일수 (거래 데이터 기준)
    SELECT
      CustomerID,
      DATE_DIFF(
        DATE(InvoiceDate),
        LAG(DATE(InvoiceDate)) OVER (
          PARTITION BY CustomerID
          ORDER BY InvoiceDate
        ),
        DAY
      ) AS interval_
    FROM `project-3-mainquest-1.modulabs_project.data`
    WHERE CustomerID IS NOT NULL
  )
  GROUP BY CustomerID
)
SELECT
u.*
pi.* EXCEPT (CustomerID)
FROM `project-3-mainquest-1.modulabs_project.user_data` AS u
LEFT JOIN purchase_intervals AS pi
ON u.CustomerID = pi.CustomerID;

```

-- 테이블 생성 확인

```

SELECT

```

*

```
FROM `project-3-mainquest-1.modulabs_project.user_data`;
```

행	CustomerID	purchase_cnt	item_cnt	recency	user_total	user_average	unique_products	average_interval
1	16881	1	600	66	432.0	432.0	1	0.0
2	15507	1	24	22	30.0	30.0	1	0.0
3	14404	1	48	17	322.1	322.1	1	0.0
4	17925	1	72	372	244.1	244.1	1	0.0
5	17956	1	1	249	12.9	12.9	1	0.0
6	17763	1	12	263	15.0	15.0	1	0.0
7	16093	1	20	106	17.0	17.0	1	0.0
8	13366	1	144	50	56.2	56.2	1	0.0
9	15488	1	72	92	76.3	76.3	1	0.0
10	15582	1	39	351	134.6	134.6	1	0.0

-- 스키마 체크 (컬럼만 빠르게 확인)

```
SELECT
  column_name
FROM `project-3-mainquest-1.modulabs_project.INFORMATION_SCHEMA.COLUMNS`
WHERE table_name = 'user_data'
ORDER BY ordinal_position;
```

행	column_name
1	CustomerID
2	purchase_cnt
3	item_cnt
4	recency
5	user_total
6	user_average
7	unique_products
8	average_interval

8.3 구매 취소 경향성

1. 취소 빈도 (cancel_frequency): 고객별로 취소한 거래의 총 횟수
2. 취소 비율 (cancel_rate): 각 고객이 한 모든 거래 중에서 취소를 한 거래의 비율

• 고객의 취소 패턴 파악하기

- 취소 빈도와 취소 비율을 계산하고 그 결과를 user_data에 통합하기
- 취소 비율은 소수점 두번째 자리까지 구하기

```
CREATE OR REPLACE TABLE `project-3-mainquest-1.modulabs_project.user_data` AS
WITH TransactionInfo AS (
  SELECT
    CustomerID,
    COUNT(DISTINCT InvoiceNo) AS total_transactions,
    COUNT(
      DISTINCT CASE
        WHEN STARTS_WITH(InvoiceNo, 'C') THEN InvoiceNo
      END
    ) AS cancel_frequency
  FROM `project-3-mainquest-1.modulabs_project.data`
  GROUP BY CustomerID
)
SELECT
  u.*,
  t.* EXCEPT (CustomerID),
  ROUND(t.cancel_frequency / t.total_transactions * 100, 2) AS cancel_rate
FROM `project-3-mainquest-1.modulabs_project.user_data` AS u
```

```
LEFT JOIN TransactionInfo AS t
ON u.CustomerID = t.CustomerID;
```

-- 결과 확인

```
SELECT
CustomerID,
total_transactions,
cancel_frequency,
cancel_rate
FROM `project-3-mainquest-1.modulabs_project.user_data`
ORDER BY cancel_rate DESC;
```

행	CustomerID	total_transactions	cancel_frequency	cancel_rate
1	16252	1	1	100.0
2	12666	1	1	100.0
3	14627	1	1	100.0
4	15823	1	1	100.0
5	16742	1	1	100.0
6	17820	2	2	100.0
7	13693	2	2	100.0
8	14777	2	2	100.0
9	12943	1	1	100.0
10	17547	1	1	100.0

- 다양한 컬럼들을 활용하여 고객의 구매 패턴과 선호도를 보다 심층적으로 이해할 수 있도록 최종적으로 **user_data**를 출력하기

```
SELECT
*
FROM `project-3-mainquest-1.modulabs_project.user_data`;
```

id	CustomerID	product_line	item_line	quantity	unit_price	unit_discount	unique_products	average_item_price	total_transactions	cancel_frequency	cancel_rate
1	14239	1	68	141	128.5	120.0	6	0.0	1	0	0.0
2	16216	1	68	30	101.7	101.7	6	0.0	1	0	0.0
3	15775	1	68	245	104.9	104.9	7	0.0	1	0	0.0
4	14096	1	45	265	152.6	152.6	7	0.0	1	0	0.0
5	15951	1	113	284	101.2	101.2	8	0.0	1	0	0.0
6	14406	1	77	119	156.8	156.8	8	0.0	1	0	0.0
7	16349	1	9	290	53.5	53.5	9	0.0	1	0	0.0
8	17044	1	105	83	233.4	233.4	9	0.0	1	0	0.0
9	12851	1	105	337	168.0	168.0	10	0.0	1	0	0.0
10	14737	1	62	274	266.6	266.6	10	0.0	1	0	0.0

회고

Keep:

- 데이터 품질을 단순 결측 여부뿐 아니라, **형식/의미 관점(StockCode 이상치)**까지 확인하려고 시도한 점은 계속 유지하고 싶다.
- 각 전처리 단계마다 **사전 확인 → 처리 → 사후 검증** 쿼리를 함께 작성한 흐름이 제이터 신뢰도를 높이는 데 도움이 되었다.
- 쿼리 가독성과 구조를 의식적으로 정리하면서 **나중에 다시 읽어도 이해 가능한 코드**를 작성하려고 한 점은 앞으로도 유지하고 싶다.

Problem:

- 전처리 과정에서 여러 단계의 CREATE OR REPLACE TABLE이 반복되며, **중간 결과를 추적하기가 다소 번거로웠다.**
- 일부 기준 (이상치 제거)은 데이터 특성을 근거로 판단했지만, **수치적 기준을 명확히 설명하지 못한 아쉬움**이 남는다.
- 쿼리 작성 자체에는 익숙해졌으나, **분석 결과를 한 문장으로 요약하는 데는 아직 부족함**을 느꼈다.

Try:

- 다음 과제에서는 전처리, 집계, 분석 단계를 **한 번에 처리하기보다 단계별로 나누어 작성**해, 전체 흐름이 더 잘 보이도록 개선해보고 싶다.
- 이상치 제거와 같은 판단이 필요한 부분에서는 **비율·분포 시각화 결과를 함께 근거로 제시**하는 연습을 해볼 것이다.

- 쿼리 결과를 단순 출력에 그치지 않고, “그래서 무엇을 알 수 있는가”를 한 줄로 정리하는 습관을 의식적으로 가져가고 싶다.